

CLASSY 103.4 FM

PADANG · THE ACTUAL RADIO

ClassyFM Mobile

API & Integration Guide

Everything the mobile app needs to talk to ClassyFM: the public JSON API, the live audio stream, and Google sign-in for chat. Written for the app developers — HTTP contracts only, no server internals.

BASE URL

<https://classyfm.co.id/api/v1>

REVISION

v1 · August 2026

FORMAT

JSON · read-oriented · cookie-free

AUDIENCE

Mobile app developers

Contents

1 Public JSON API

- Conventions — envelopes, errors, caching, CORS, pagination
- Content endpoints — programs, broadcasters, news, podcasts, about
- Feed & live endpoints — now-playing, schedule, TikTok, config, home
- Connect chat API
- Authentication & rate limits
- Object reference

2 Direct Shoutcast streaming (alternative to the JSON API)

3 Mobile Google sign-in integration

Values you'll need

API base URL	<code>https://classyfm.co.id/api/v1</code>
Audio stream URL	<code>https://classyfm.co.id/stream</code>
Google Web client id	<code>1021202175550-650knthsj9nnfgnia22hs8u2pmhgdmv8.apps.googleusercontent.com</code>

The stream URL is also returned live by GET `/api/v1/config` as `stream_url` — read it from there rather than hard-coding.

SECTION 1

Public JSON API

The public, read-oriented JSON API for the ClassyFM mobile app. All endpoints are mounted under `/api/v1`.

- **Base URL:** `https://classyfm.co.id/api/v1`
- **Format:** JSON (Content-Type: `application/json; charset=utf-8`)
- **Orientation:** read-only, **cookie-free** — the only writes are the Connect chat endpoints, which authenticate with a Bearer token rather than a session cookie.
- **CORS:** permissive by default, so the app (or a browser) can call it directly.

LEGACY POLLING ENDPOINTS

The website's floating widgets poll a few older, unversioned endpoints that predate `/api/v1`: `/api/nowplaying`, `/api/schedule/today`, `/api/schedule/current`, `/api/tiktok/live`, and `/api/connect/messages`. They return the same data as their `/api/v1` counterparts and exist for the web widgets — **new clients should use `/api/v1`**.

Conventions

Response envelopes

SHAPE	USED BY	BODY
List	roster / list endpoints	<code>{ "data": [...] }</code>
Paginated	GET <code>/news?source=...</code> , GET <code>/podcasts</code>	<code>{ "data": [...], "meta": { "page": 1, "total_pages": 3, "total": 27 } }</code>
Object	detail & aggregate endpoints	the object directly (no wrapper)

Errors

Errors return `{ "error": "message" }` with the relevant HTTP status:

STATUS	MEANING
400	Bad or empty request body, invalid path id
401	Authentication required / invalid Google or Bearer token
403	Banned from chat, or moderation (admin) access required
404	Resource not found, or the feature is not configured
429	Rate limit exceeded — see Rate limits
500	Server / database error
503	Chat temporarily unavailable

The `429` response carries a `Retry-After: 60` header with the usual JSON body:
`{ "error": "too many requests, try again later" }`.

Caching

Each response carries a `Cache-Control` header:

- `public, max-age=60` — slowly-changing content (programs, news, podcasts, about, home, config).
- `no-store` — live or per-user data (now-playing, schedule state, TikTok live, all chat endpoints).

CORS

Applied to every `/api/v1/*` route:

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Max-Age: 86400
```

Preflight `OPTIONS` requests short-circuit with `204 No Content`.

Pagination & filtering

Pagination applies only to GET `/news?source=...` and GET `/podcasts`:

- `page` — 1-based, defaults to `1` (values `< 1` are treated as `1`).
- Page size is fixed at `12`.
- `meta.total_pages` is at least `1` even when empty.

URLs & images

All image and link fields are returned as **absolute URLs** (resolved against the site origin, `https://classyfm.co.id`). Already-absolute URLs — e.g. aggregated news thumbnails — pass through unchanged.

Degraded mode

If the backend is temporarily unavailable, **list** endpoints degrade gracefully to `{ "data": [] }` and **detail** endpoints return `404` — they never `500`. Handle empty lists and 404s defensively.

Content endpoints

All are `GET` and cached `public, max-age=60`.

`GET /api/v1/programs`

Active program roster, each flagged with whether it is on air now. See the [program](#) object.

```
{ "data": [
  {
    "title": "Morning Show",
    "slug": "morning-show",
    "description": "...",
    "image_url": "https://classyfm.co.id/uploads/abc.jpg",
    "url": "https://classyfm.co.id/program/morning-show",
    "on_air": true
  }
] }
```

GET /api/v1/programs/{slug}

One program with its full weekly schedule and broadcasters. **404** if the slug is unknown or the program is inactive.

```
{
  "title": "Morning Show",
  "slug": "morning-show",
  "description": "...",
  "image_url": "https://classyfm.co.id/uploads/abc.jpg",
  "url": "https://classyfm.co.id/program/morning-show",
  "on_air": true,
  "schedule": [
    { "from_day": 1, "to_day": 5, "start_time": "06:00", "end_time": "09:00", "host": "Anda, Yeni" },
    ...
  ],
  "broadcasters": [ /* broadcaster objects */ ]
}
```

`schedule[]` items are [scheduleGroup](#) objects; `broadcasters[]` are [broadcaster](#) objects.

GET /api/v1/broadcasters

Active broadcaster roster. `on_air` is `true` if any program the broadcaster presents is airing now.

```
{ "data": [ /* broadcaster objects */ ] }
```

GET /api/v1/broadcasters/{slug}

One broadcaster plus the programs they present. **404** if unknown.

```
{
  "name": "Anda",
  "slug": "anda",
  "role": "Announcer",
  "photo_url": "https://classyfm.co.id/uploads/anda.jpg",
  "bio": "...",
  "url": "https://classyfm.co.id/broadcasters/anda",
  "on_air": false,
  "programs": [ /* program objects */ ]
}
```

GET /api/v1/news

Two modes. **Without** `source` (or an unrecognized one) — grouped preview, up to 6 items per source group:

```
{ "data": [
  { "source": "klikpositif", "label": "KlikPositif", "items": [ /* news items */ ] },
  { "source": "katasumbar", "label": "KataSumbar", "items": [ ... ] },
  { "source": "hot_release", "label": "Hot Release", "items": [ ... ] },
  { "source": "youtube", "label": "YouTube", "items": [ ... ] }
] }
```

With a valid `source` — that source's items, paginated:

QUERY PARAM	NOTES
source	One of <code>youtube</code> , <code>klikpositif</code> , <code>katasumber</code> , <code>hot_release</code> . Invalid values fall back to grouped mode.
page	1-based, page size 12.

```
{ "data": [ /* news items */ ], "meta": { "page": 1, "total_pages": 4, "total": 42 } }
```

Items are `newsItem` objects. Note: `hot_release` items link to the on-site `/news/{slug}` ; other (aggregated) sources keep their external `url` .

GET `/api/v1/news/{slug}`

One `hot_release` article, with split paragraphs, a mid-article image gallery, and related articles. `404` if unknown (only `hot_release` articles have on-site detail).

```
{
  "source": "hot_release",
  "source_label": "Hot Release",
  "title": "...",
  "slug": "...",
  "excerpt": "...",
  "image_url": "https://...",
  "url": "https://classyfm.co.id/news/...",
  "published_at": "2026-08-09T10:00:00Z",
  "is_featured": true,
  "paragraphs": ["...", "..."],
  "gallery_index": 3,
  "middle_images": ["https://...", "https://..."],
  "related": [ /* news items */ ]
}
```

`gallery_index` is the paragraph index after which the `middle_images` gallery should be inserted.

GET `/api/v1/podcasts`

Published podcasts, newest first, paginated.

QUERY PARAM	NOTES
series	Optional series slug filter. Invalid slugs are ignored (returns all).
page	1-based, page size 12.

```
{ "data": [ /* podcast objects */ ], "meta": { "page": 1, "total_pages": 2, "total": 18 } }
```

GET `/api/v1/podcasts/{slug}`

One podcast with its series name and broadcasters. `404` if unknown.

```
{
  "title": "...",
  "slug": "...",
  "description": "...",
  "spotify_url": "https://open.spotify.com/...",
  "thumb_url": "https://...",
  "series_name": "...",
  "url": "https://classyfm.co.id/podcast/...",
  "broadcasters": [ /* broadcaster objects */ ]
}
```

GET /api/v1/podcast-series

Podcast series list (for a filter picker feeding `?series=`).

```
{ "data": [ { "name": "...", "slug": "..." } ] }
```

GET /api/v1/about

About-page content: banner, text segments, and a broadcaster preview (max 8).

```
{
  "banner": {
    "media_type": "video",
    "image_url": "https://...",
    "video_url": "https://youtu.be/...",
    "embed_url": "https://www.youtube.com/embed/..."
  },
  "segments": [ { "segment": "vision", "title": "...", "body": "..." } ],
  "broadcasters": [ /* broadcaster objects */ ]
}
```

`banner.embed_url` is present only when `media_type` is `"video"` and the video URL is a resolvable YouTube link.

Feed & live endpoints

All are GET. `now-playing`, `schedule/*`, and `tiktok/live` are `no-store`; `config` and `home` are cached `public`, `max-age=60`.

GET /api/v1/now-playing

Stream now-playing metadata, plus the on-air program when the stream is live. Cover art is enriched server-side, so prefer this over reading the raw stream (see Section 2).

```
{
  "artist": "...",
  "song": "...",
  "has_song": true,
  "cover_url": "https://...",
  "live": true,
  "program": { /* scheduleRow, present only when live and a program is on air */ }
}
```

See [scheduleRow](#). The listener count is deliberately not exposed here.

GET /api/v1/schedule/today

Today's full schedule with live on-air / progress state.

```
{ "data": [ /* scheduleRow objects */ ] }
```

GET /api/v1/schedule/current

The currently on-air program as a single [scheduleRow](#) object, or `{ "on_air": false }` when nothing is airing.

GET /api/v1/tiktok/live

```
{ "live": true, "title": "..." }
```

GET /api/v1/config

App bootstrap: station identity, stream URL, social links, and whether chat sign-in is available.

```
{
  "station": { "name": "Classy 103.4 FM", "slogan": "..." },
  "stream_url": "https://classyfm.co.id/stream",
  "social": { "instagram": "https://...", "youtube": "https://..." },
  "connect": { "enabled": true }
}
```

`connect.enabled` is `true` only when Google sign-in is configured (see Section 3). `social` keys are platform names.

GET /api/v1/home

Aggregate home-screen feed in a single request — mirrors the web landing page.

```
{
  "hero": [ /* heroSlide objects */ ],
  "on_air": { /* scheduleRow, or null */ },
  "programs": [ /* program objects, up to 6 */ ],
  "news": [ /* newsGroup objects, up to 4 items each */ ]
}
```

See [heroSlide](#).

Connect chat API

The Connect chatroom over JSON, mounted under `/api/v1/connect`. **Reads are open; writes require a Bearer token** obtained by exchanging a Google ID token (see Section 3). All chat responses are `no-store`.

GET /api/v1/connect/messages OPEN

Poll chat messages.

QUERY PARAM	NOTES
<code>since</code>	Optional message id. With it, returns up to 200 messages newer than that id (delta poll). Without it, returns the 50 most recent.


```
{ "messages": [ /* chatMessage objects */ ] }
```

POST **/api/v1/connect/session** **OPEN** 30 / MIN

Exchange a native Google ID token for a Connect session token. The app obtains the ID token via the platform's own Google sign-in (so Google OAuth never runs in a WebView), passing the website's Google client id as its `serverClientId`.

Request:

```
{ "id_token": "<google-id-token>" }
```

Response:

```
{
  "token": "<connect-session-token>",
  "user": { "id": 42, "name": "...", "avatar": "https://...", "is_admin": false }
}
```

The `id_token` is verified server-side against Google (checking audience and issuer), then the chat user is upserted and the admin badge recomputed. Send `token` as `Authorization: Bearer <token>` on subsequent authenticated calls.

Errors: `404` if chat login is not configured, `400` if `id_token` is missing, `401 invalid Google token`, `500` on failure. The request body is capped at 16 KiB.

GET **/api/v1/connect/me** **BEARER**

Returns the chat identity behind the token.

```
{ "user": { "id": 42, "name": "...", "avatar": "https://...", "is_admin": false } }
```

POST **/api/v1/connect/messages** **BEARER** 20 / MIN

Post a chat message. The body is sanitized, trimmed, and capped at **1000 characters**.

Request:

```
{ "body": "Hello!" }
```

Response — the stored message, so the app can render it optimistically:

```
{ "message": { "id": 101, "user_id": 42, "name": "...", "avatar": "...", "is_admin": false, "body": "He"
```

Errors: `401` if unauthenticated, `403 your account is blocked from chat` if banned, `400` if the body is empty/invalid, `503` if chat is unavailable. Body capped at 16 KiB.

POST **/api/v1/connect/messages/{id}/delete** **BEARER + ADMIN**

Soft-delete a message. Returns `{ "ok": true }`. **Errors:** `401` if unauthenticated, `403 moderation access required` if not an admin, `400 invalid id`.

POST**/api/v1/connect/users/{id}/ban****BEARER + ADMIN**

Block a chat user from posting. Returns `{ "ok": true }`. Same auth / id guards as delete.

Authentication

Chat writes use a self-contained, signed **Bearer token** — no server-side session row, no cookie. The full mobile sign-in walkthrough is **Section 3**; in short:

1. **Obtain a token.** The native app signs in with Google on-device and receives a Google **ID token**. It `POST`s that to `/api/v1/connect/session`, which verifies it with Google (enforcing that the audience equals the configured Google client id and the issuer is Google), upserts the chat user, and returns a **Connect session token**.
2. **Use the token.** Send it on authenticated endpoints: `Authorization: Bearer <connect-session-token>`
3. **Token properties.**
 - Carries only the chat user id and an expiry, signed by the server.
 - **TTL: 30 days.**
 - On each request the identity (name, avatar, admin / ban status) is re-read server-side, so bans and role changes take effect immediately without re-issuing the token.
 - Enforcement is per-endpoint: `401` when the token is missing / invalid, `403` when a moderation action requires an admin the token's user is not.

Rate limits

Per client IP, fixed one-minute window:

ENDPOINT	LIMIT
<code>POST /api/v1/connect/session</code>	30 / minute
<code>POST /api/v1/connect/messages</code>	20 / minute

Exceeding a limit returns `429` with a `Retry-After: 60` header and the standard JSON error body `{ "error": "too many requests, try again later" }`. The delete and ban endpoints are not rate-limited.

Object reference

Fields marked *(optional)* are omitted from the JSON when empty.

program

FIELD	TYPE	NOTES
<code>title</code>	string	
<code>slug</code>	string	
<code>description</code>	string	<i>(optional)</i>
<code>image_url</code>	string	<i>(optional)</i> absolute URL
<code>url</code>	string	on-site program URL
<code>on_air</code>	bool	airing now

scheduleGroup

FIELD	TYPE	NOTES
from_day	int	0 = Sunday ... 6 = Saturday
to_day	int	end of the day range (same numbering)
start_time	string	HH:MM
end_time	string	HH:MM
host	string	<i>(optional)</i> effective broadcaster set, e.g. "Anda, Yeni"

broadcaster

FIELD	TYPE	NOTES
name	string	
slug	string	
role	string	<i>(optional)</i>
photo_url	string	<i>(optional)</i> absolute URL
bio	string	<i>(optional)</i>
birth_place	string	<i>(optional)</i>
birth_date	string	<i>(optional)</i>
instagram	string	<i>(optional)</i>
twitter	string	<i>(optional)</i>
facebook	string	<i>(optional)</i>
url	string	on-site broadcaster URL
on_air	bool	presenting an airing program now

newsItem

FIELD	TYPE	NOTES
source	string	youtube klikpositif katasumbar hot_release
source_label	string	human-readable source name
title	string	
slug	string	<i>(optional)</i> present for hot_release
excerpt	string	<i>(optional)</i>
image_url	string	<i>(optional)</i>
url	string	on-site for hot_release, external otherwise
published_at	string	RFC 3339 timestamp
is_featured	bool	<i>(optional)</i>

newsGroup

FIELD	TYPE	NOTES
source	string	source key
label	string	display label
items	newsItem[]	

podcast

FIELD	TYPE	NOTES
title	string	
slug	string	
description	string	(optional)
spotify_url	string	(optional)
thumb_url	string	(optional) absolute URL
series_name	string	(optional)
broadcaster	string	(optional) list-view only
url	string	on-site podcast URL

scheduleRow

FIELD	TYPE	NOTES
title	string	program title
slug	string	(optional)
host	string	(optional)
image	string	(optional) absolute URL
start_time	string	HH:MM
end_time	string	HH:MM
on_air	bool	
progress	int	0-100, % through the slot
ended	bool	slot already finished today

heroSlide

FIELD	TYPE	NOTES
href	string	<i>(optional)</i> link target
target_blank	bool	<i>(optional)</i> open in new tab
image_url	string	<i>(optional)</i>
badge_label	string	<i>(optional)</i>
title	string	
excerpt	string	<i>(optional)</i>
date	string	<i>(optional)</i> RFC 3339 timestamp

connectUser

FIELD	TYPE	NOTES
id	int	chat user id
name	string	
avatar	string	<i>(optional)</i>
is_admin	bool	

chatMessage

FIELD	TYPE	NOTES
id	int	message id (use as <code>since</code> cursor)
user_id	int	author's chat user id (target of delete / ban)
name	string	author name
avatar	string	author avatar URL
is_admin	bool	author is a moderator
body	string	sanitized message text
time	string	HH:MM, station timezone

SECTION 2

Direct Shoutcast streaming

An alternative to the JSON API for the live audio: connect straight to the Shoutcast origin to play the stream and read now-playing metadata, with no dependency on `/api/v1`.

The live audio is served by a standard **Shoutcast v2** server. The app can play it and read its metadata directly. This is the lowest-latency path for the **audio itself**, and the recommended way to actually play the stream. For the **song/artist metadata**, however, the JSON API's `GET /api/v1/now-playing` is preferable — it adds cover art and the on-air program, is CORS-safe, and is cached — so a common split is: **play the audio direct, read metadata from the API**.

READ THE STREAM URL FROM CONFIG

The convenience URL `https://classyfm.co.id/stream` is a stable public entry point (a reverse proxy in front of the Shoutcast origin) and is what `GET /api/v1/config` returns as `stream_url`. Prefer reading it from `/config` at startup rather than hard-coding, so the endpoint can move without an app update. The direct origin details below are for when you want to bypass the proxy and API entirely.

Audio stream

PROPERTY	VALUE
Public URL (recommended)	<code>https://classyfm.co.id/stream</code>
Origin URL (direct)	<code>https://c4.siar.us:10340/stream.mp3</code>
Server	Shoutcast v2
Codec / bitrate	MP3, 128 kbps
Origin base (for metadata)	<code>https://c4.siar.us:10340</code>

Any standard streaming player (ExoPlayer on Android, AVPlayer on iOS, or an HTML5 `<audio>` element) can play the URL directly — it is a continuous MP3 stream, so treat it as live (no seek, no duration).

Now-playing metadata (direct)

GET `https://c4.siar.us:10340/played.html`

Returns an HTML "Stream History" table. The **current song is the top data row**; the second cell holds the title as `"Artist - Title"`, or literally `"Empty Title"` when the stream carries no track metadata. Split on the first `" - "` to separate artist and title.

ALTERNATIVE: IN-BAND ICY METADATA

Instead of polling `played.html`, a player can read the title inline from the audio stream itself by sending the request header `Icy-MetaData: 1` when opening the stream. The server then interleaves `StreamTitle='...'` metadata blocks into the audio. Most native players (ExoPlayer, AVPlayer) expose this as ICY / timed metadata callbacks — usually the cleanest option on mobile since it needs no extra polling.

Live status (direct)

GET `https://c4.siar.us:10340/stats?json=1`

Returns JSON server stats. Use `streamstatus` to tell whether the station is broadcasting live:

```
{
  "streamstatus": 1,           // 1 = live/broadcasting, 0 = down
  "currentlisteners": 128,
  "songtitle": ""             // empty on this server - use played.html or ICY for the song
}
```

Note `songtitle` is empty on this server, so it can't be used for now-playing — rely on `played.html` or ICY metadata for the song.

Which path to use

	DIRECT SHOUTCAST	JSON API (/API/V1/NOW-PLAYING)
Audio playback	✓ lowest latency, recommended	— (metadata only)
Song / artist	✓ (<code>played.html</code> or ICY)	✓ same data
Cover art	✗ not provided	✓ enriched server-side
On-air program join	✗	✓ <code>program</code> field when live
Live flag	✓ <code>streamstatus</code>	✓ <code>live</code>
CORS-safe / cached	✗ raw origin	✓ CORS + 12 s cache
Listener count	exposes <code>currentlisteners</code>	deliberately hidden

RECOMMENDATION

Play the audio directly from the stream URL; get song, cover art, and the on-air program from `GET /api/v1/now-playing`. Fall back to `played.html` / ICY metadata only if you specifically want to avoid any dependency on the API.

SECTION 3

Mobile Google sign-in

How the app signs a user in with Google natively and exchanges that for a Connect chat token. Google OAuth runs on-device through the platform SDK — never in a WebView.

The one value you need

Google Web client id 1021202175550-650knthsj9nnfgnia22hs8u2pmhgdmv8.apps.googleusercontent.com

This is a **Web application** OAuth client from ClassyFM's Google Cloud project — the same one the website uses. The app must obtain a Google **ID token** whose audience (`aud`) equals this string. There is **no client secret** on the mobile side; the app never needs it.

The flow

1 Configure native Google sign-in

Add ClassyFM's Google project client id above as your `serverClientId` (Android — Credential Manager / Google Identity Services) or as the `serverClientId` / requested audience (iOS — Google Sign-In SDK). This makes Google mint an ID token whose `aud` is that Web client id, which is what the server verifies against. You will also register the app's own platform OAuth client (Android package + SHA-1, or iOS bundle id) in the same Google project.

2 Sign the user in on-device

Trigger the platform's Google sign-in UI. On success you receive a Google **ID token** (a JWT). Do this through the native SDK — never open Google's OAuth page inside a WebView.

3 Exchange the ID token for a Connect token

`POST /api/v1/connect/session` with `{ "id_token": "<google-id-token>" }`. The server validates the token with Google (audience must equal the client id above; issuer must be Google), upserts the chat user, and returns:

```
{ "token": "<connect-session-token>", "user": { "id": 42, "name": "...", "avatar": "https://...", "is_admin": false } }
```

Errors: `401 invalid Google token` if the audience/issuer don't match, `400` if `id_token` is missing, `404` if chat sign-in isn't configured on the server.

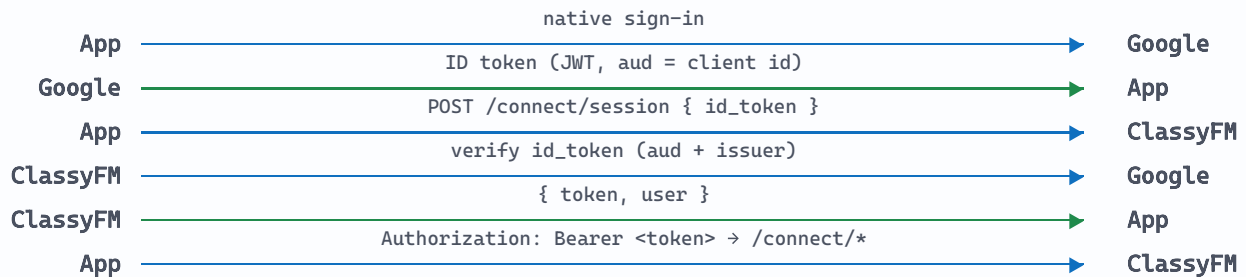
4 Store & use the Connect token

Persist `token` securely (Keychain / Keystore) and send it as `Authorization: Bearer <token>` on every authenticated Connect call — `GET /connect/me`, `POST /connect/messages`, and the admin delete / ban actions.

5 Handle expiry & state changes

The Connect token's **TTL is 30 days**. On a `401`, silently re-run the Google sign-in and re-exchange. The server re-reads name, avatar, admin, and ban status on every request, so a ban or role change applies immediately — a banned user gets `403` on `POST /connect/messages` without needing a new token.

Sequence



The app talks to Google only for sign-in; ClassyFM independently re-verifies the ID token with Google before issuing its own token.

Feature gate

Before showing any sign-in or chat UI, check `GET /api/v1/config` → `connect.enabled`. It is `true` only when Google sign-in is configured on the server. When it is `false`, hide the Connect chat entirely — the `/connect/session` endpoint will return `404`.

COMMON PITFALLS

Wrong audience. If sign-in succeeds but `/connect/session` returns `401 invalid Google token`, the ID token's `aud` isn't the Web client id above — double-check `serverClientId` (Android) / `audience` (iOS).

Using an access token instead of an ID token. The endpoint expects the Google **ID token** (a JWT), not an OAuth access token.

WebView sign-in. Run Google sign-in through the native SDK; Webview-based OAuth is blocked by Google and unnecessary here.